Membrane Computation and Max-SMT in Rust

November 6, 2025

Contents

1 Membrane Computation Overview

Membrane computation is a parallel computational framework where:

- **Membranes** represent candidate solutions.
- **Rules** transform membrane states (variables) iteratively.
- **Environment** defines constraints.
- **Evaluation** determines which membranes survive.

1.1 Binary SMT Approach

Membranes are either **alive** if constraints are satisfied, or **dead** otherwise.

1.2 Max-SMT / Optimization Approach

Each membrane is assigned a **score** based on how many constraints it satisfies. Soft constraints contribute partially, while hard constraints can still eliminate membranes.

$$score(M) = \sum_{i=1}^{n} w_i \cdot sat(c_i, M)$$

where w_i is the weight of constraint c_i and $sat(c_i, M)$ is the degree of satisfaction (0..1).

2 Rust Implementation

2.1 Membrane and Constraint Structures

```
use serde::Serialize;
use std::collections::HashMap;

#[derive(Serialize)]
struct Membrane {
    id: usize,
    variables: HashMap<String, f64>,
    score: f64,
}

#[derive(Clone)]
struct Constraint {
    name: String,
    weight: f64,
    limit: f64,
    vars: Vec<String>,
}
```

```
impl Constraint {
    fn satisfaction(&self, vars: &HashMap<String, f64>) -> f64 {
        let sum: f64 = self.vars.iter().map(|v| *vars.get(v).
           unwrap_or(&0.0)).sum();
        if sum <= self.limit {
            1.0
        } else {
            self.limit / sum
        }
    }
}
```

2.2 Evaluate Membrane Scores

```
fn evaluate_membrane(m: &mut Membrane, constraints: &[Constraint]) {
    let mut total_score = 0.0;
    for c in constraints {
        total_score += c.weight * c.satisfaction(&m.variables);
    }
    m.score = total_score;
}
```

2.3 Generate and Rank Membranes

```
fn main() -> std::io::Result<()> {
    let constraints = vec![
        Constraint { name: "c1".to_string(), weight: 1.0, limit:
           10.0, vars: vec!["x".to_string(), "y".to_string()] },
        Constraint { name: "c2".to_string(), weight: 2.0, limit:
           5.0, vars: vec!["x".to_string()] },
    ];

    let mut membranes = vec![];
    for i in 0..5 {
        let mut vars = HashMap::new();
        vars.insert("x".to_string(), i as f64 + 1.0);
        vars.insert("y".to_string(), (5 - i) as f64);
        let mut m = Membrane { id: i, variables: vars, score: 0.0 };
        evaluate_membrane(&mut m, &constraints);
        membranes.push(m);
    }

    membranes.sort_by(|a, b| b.score.partial_cmp(&a.score).unwrap())
       ;
```

```
    let json = serde_json::to_string_pretty(&membranes)?;
    std::fs::write("membranes.json", json)?;
    println!("JSON saved to membranes.json");

    Ok(())
}
```

2.4 Example D3.js JSON Output

```
[
  {"id":0,"variables":{"x":1.0,"y":5.0},"score":3.9},
  {"id":1,"variables":{"x":2.0,"y":4.0},"score":4.2}
]
```

3 Mutation Rules for Membranes

```
use rand::Rng;

impl Membrane {
    fn mutate(&mut self, mutation_rate: f64, mutation_range: f64) {
        let mut rng = rand::thread_rng();
        for value in self.variables.values_mut() {
            if rng.gen::<f64>() < mutation_rate {
                let delta = rng.gen_range(-mutation_range..
                   mutation_range);
                *value += delta;
            }
        }
    }
}
```

3.1 Using Mutation in Evolution Loop

```
for generation in 0..10 {
    for m in membranes.iter_mut() {
        evaluate_membrane(m, &constraints);
    }

    membranes.sort_by(|a, b| b.score.partial_cmp(&a.score).unwrap())
       ;

    for m in membranes.iter_mut().skip(1) {
```

```
        m.mutate(0.5, 1.0);
    }
}
```

4 Visualization Notes

- Export membrane scores each generation to JSON.
- Color nodes by score (heatmap/gradient).
- Optional: animate over generations to see optimization progress.

5 Summary

- Membrane computing provides a **parallel framework** for Max-SMT problems.
- Ranking membranes by score supports **soft constraints** and **optimization**.
- Mutation rules allow **exploration of solution space**.
- D3.js JSON output enables **interactive visualization**.

This framework can be extended to:

- Non-linear SMT constraints
- Larger populations
- Advanced selection and mutation strategies
- Real-time interactive visualization